

Workshop 3: Datenmodell und Validierung in Google Sheets

Legal History Hub · Max-Planck-Institut für Rechtsgeschichte und Rechtstheorie

Mag. Christian Steiner MA · Digital Humanities Craft OG · 2026

Folien erstellt mit Claude Code in VS Code, gerendert als HTML mit Marp. Quelle: Markdown-Datei im Repo, versionierbar wie der Rest des Projekts. Der Workflow ist selbst Teil dessen, was ihr lernen werdet.

Agenda

1. **Wide, Long, Tidy** – Warum unser Sheet so aussieht, wie es aussieht (45 min)
2. **Datenmodell mit Claude Code** – Lesen, adaptieren, validieren (45 min)

Pause (10-15 min)

3. **Google Sheets als CMS** – Tables, Dropdowns, Ansichten (45 min)
4. **Skills und Plugins** – Claude Code erweitern (30 min)
5. **Zusammenfassung und Ausblick** (15 min)

Begriffe, die heute fallen werden (1/2)

Begriff	Kurz
FAIR	Findable, Accessible, Interoperable, Reusable (Datenqualitäts-Prinzip)
PID	Persistent Identifier, z.B. ORCID, GND, ROR, Wikidata-Q-Nummer
Normdatei / Authority	zentrale Liste mit Namen und ihren IDs
Foreign Key	Verweis in eine andere Tabelle (hier: <code>project_id</code>)
Junction Table	Tabelle, die eine Many-to-Many-Beziehung auflöst
Many-to-Many	Ein Projekt hat viele Personen, eine Person viele Projekte

Begriffe, die heute fallen werden (2/2)

Begriff	Kurz
Singleton	Feld mit genau einem Wert pro Entität (z.B. Titel)
Source of Truth	die eine Stelle, an der der richtige Wert steht
Wide / Long / Tidy	Daten-Formate (kommen gleich ausführlich)
1NF	First Normal Form: in jeder Zelle steht genau ein Wert
Enum	Geschlossene Werteliste (z.B. status: <code>active</code> , <code>completed</code> , <code>planned</code>)
Spill Range	Formel, die automatisch mehrere Ergebniszeilen ausfüllt

Diese beiden Folien bleiben als Referenz. Beim Zurückblättern findet ihr hier die Definitionen.

Block 1: Wide, Long, Tidy

Das Pilot-Experiment: der erste Anlauf

Zwischen WS2 und WS3 hat **Polina** ein eigenes Datenmodell als Experiment gebaut. → [Polinas Pilot-Sheet](#)

6 Tabs, relational normalisiert: eigene Tabs für Projekte, Personen, Organisationen, Themen, Keywords und Regionen.

Konzeptuell sauber: eigene IDs (`pers-003` , `subj-001`), ORCIDs, GND-Nummern. Das ist relationales Denken und grundsätzlich richtig.

Was am Pilot-Modell richtig war

	A	B	C	D	E	F	G	H	I	J	K
1	person_id	name	orcid_id	orcid_status							
2	pers-001	Wagner, Andreas	0000-0003-1835-1653	confir... ▼							
3	pers-002	Birr, Christiane		none ▼							
4	pers-003	Duve, Thomas	0000-0002-1658-4173	confir... ▼							
5	pers-004	Lutz-Bachmann, Matth	0009-0006-4006-8773	confir... ▼							
6	pers-005	König, Florian	0009-0006-2438-0513	confir... ▼							
7	pers-006	Rico Carmona, Cindy	0000-0001-5095-1793	confir... ▼							
8	pers-007	Solonet, Polina	0000-0001-5223-1220	confir... ▼							
9	pers-008	Hugel, Marie-Astrid		none ▼							
10	pers-009	Schweighöfer, Stefan	0009-0001-6421-736X	confir... ▼							
11	pers-010	Mejía, Pilar	0000-0002-2690-1734	confir... ▼							
12	pers-011	Soler, Ana Isabel		none ▼							
13	pers-012	Aguiar, Nádia dos Santos		none ▼							
14	pers-013	Goncalves, Larissa		none ▼							
15	pers-014	Collin, Peter		none ▼							
16	pers-015	Wolf, Johanna	0000-0001-6364-8902	confir... ▼							
17	pers-016	Ebbertz, Matthias	0000-0001-6440-2453	confir... ▼							
18	pers-017	Vesper, Tim-Niklas		none ▼							
19	pers-018	Michel, Lisa		none ▼							
20	pers-019	Gödde, Ben		none ▼							
21	pers-020	Wolf, Paulina		none ▼							
22	pers-021	Damler, Daniel		none ▼							
23	pers-022	Egío García, José Luis	0000-0002-9256-8490	confir... ▼							
24	pers-023	Schüssler, Rudolf		none ▼							
25				▼							
26				▼							
27				▼							
28				▼							

Wo es hakt: Separator-Listen

	F	G	H	I	J	K	L
1	description_sh	description_sh	description_sh	description_de	description_en	description_es	creator
2	e S. Una edición digit	A digital edition c	Eine digitale Aus	Die digitale Quel	The Digital Collection of Sources contain	La Colección Digital de Fuentes contiene textos completos y copias digitale	pers-003 PI;pers
3	nón Eine digitale Que	A digital source €	Una edición digit	In der Quellensa	In the source collection, sets of rules are	En la colección de fuentes se editan conjuntos de normas que muestran la	pers-014 PI;pers
4	listó Ein historisches	A historical dictio	Un diccionario hi	Historisches Wö	The Historical Dictionary of Canon Law in	Diccionario Histórico de Derecho Canónico en Hispanoamérica y Filipinas (	pers-003 PI;pers
5	hán Gonzalo Fernán	Gonzalo Fernán	Gonzalo Fernán	Er war das Idol v	He was the idol of Alexander von Humbo	Fue el ídolo de Alexander von Humboldt, quien lo admiraba como el «Plinio	pers-021 PI
6	iad Die Dissertation	This dissertation	La tesis doctoral	Die Dissertation	This dissertation examines the regulation	Esta tesis examina la regulación de las relaciones laborales y las condicon	pers-016 PI
7							
8							
9							
10							
11							
12							
13							
14							
15							
16							
17							
18							
19							
20							
21							
22							
23							
24							
25							
26							
27							
28							

Warum ist das ein Problem?

In der Datenbanktheorie gibt es eine Regel namens **1. Normalform (1NF)**:

„An entry in a table is not decomposable.“
– Edgar F. Codd, *A Relational Model of Data for Large Shared Data Banks*, 1970.

Codd war IBM-Forscher und hat die relationalen Datenbanken erfunden. Seine Regel heißt:
In einer Zelle steht genau ein Wert. Keine Listen, keine Trennzeichen.

	J	K	L	M	N
1	on_en	description_es	creator	subject	keywords_de
2	al Collection of Sources contain	La Colección Digital de Fuentes contiene textos completos y copias digitale	pers-003 PI;pers	subj-001;subj-002;subj-003	kooperation;digitale-geisteswissenschaften;quelle
3	urce collection, sets of rules are	En la colección de fuentes se editan conjuntos de normas que muestran la	pers-014 PI;pers	subj-004	digital-humanities;sonderordnungen
4	orical Dictionary of Canon Law in	Diccionario Histórico de Derecho Canónico en Hispanoamérica y Filipinas (	pers-003 PI;pers	subj-001	iberische-welten;normativität;religion
5	he idol of Alexander von Humboldt	Fue el ídolo de Alexander von Humboldt, quien lo admiraba como el «Plinio	pers-021 PI	subj-005	transmedia-historytelling;rechtshistorische-praxis
6	ertation examines the regulation	Esta tesis examina la regulación de las relaciones laborales y las condicon	pers-016 PI	subj-004	sonderordnungen
7					
8					
9					
10					
11					
12					
13					

Warum viele Spalten auch nicht helfen

project_id	person_1	role_1	person_2	role_2	...	person_15
proj-001	Duve	PI	L-B	PI	...	

Wie viele Spaltenpaare reichen? Bei 3 Personen sind 12 Spalten leer. Bei 20 müsste man die Tabellenstruktur ändern.

Eine Tabelle voller leerer Zellen, endloses Scrollen nach rechts, starre Obergrenze. In der DB-Theorie: **Repeating Groups** (Codd 1970).

Drei Formate im Vergleich

Format	Wie es aussieht	Problem
Wide + Separator-Listen	<code>creator = "Duve, T.; L-B, M."</code> in einer Zelle	Sheets sieht nur Text, kein Filtern/Zählen
Relational normalisiert	Eigene Personen-Tabelle, nur IDs in Projekten	Sheets kann keine Joins, Menschen müssen mental verknüpfen
Hybrid (wide + long)	Wide für Singletons, Long-Tab für Beziehungen	Redundanz in Titel-Spalten, dafür Sheets-tauglich

Das Pilot-Modell war Format 2 (plus Separator-Listen). Wir bauen heute Format 3.

Warum nicht Format 2? Weil Sheets Nutzer-Oberfläche ist, nicht Datenbank. Menschen lesen, klicken, filtern. Joins gibt es nicht.

Die Lösung: eigener Tab im Long-Format

project_id	person	role
proj-001	Duve, Thomas	PI
proj-001	Lutz-Bachmann, M.	PI
proj-001	König, Florian	researcher
proj-002	Collin, Peter	PI

Eine Zeile pro Person-Projekt-Rolle. Das nennt man **Long-Format**.

„Each variable is a column, each observation is a row.“ – Hadley Wickham, *Tidy Data*, 2014.

Der Tab selbst ist eine **Junction Table**: eine Tabelle, die eine Many-to-Many-Beziehung auflöst.

So sieht das in unserem Sheet aus

	A	B	C	D	E	F	G	H	I
1	project_id ▾	title_de ▾	person ▾	role ▾					
2	proj-001	Die Schule von Salamanca. Eine digita	Duve, Thomas ▾	PI ▾					
3	proj-001	Die Schule von Salamanca. Eine digita	Lutz-Bachmann, Matthias ▾	PI ▾					
4	proj-001	Die Schule von Salamanca. Eine digita	König, Florian ▾	researcher ▾					
5	proj-001	Die Schule von Salamanca. Eine digita	Birr, Christiane ▾	researcher ▾					
6	proj-001	Die Schule von Salamanca. Eine digita	Rico Carmona, Cindy ▾	researcher ▾					
7	proj-001	Die Schule von Salamanca. Eine digita	Solonets, Polina ▾	researcher ▾					
8	proj-001	Die Schule von Salamanca. Eine digita	Wagner, Andreas ▾	researcher ▾					
9	proj-001	Die Schule von Salamanca. Eine digita	Hugel, Marie-Astrid ▾	researcher ▾					
10	proj-001	Die Schule von Salamanca. Eine digita	Schweighöfer, Stefan ▾	project-coordinator ▾					
11	proj-002	Nichtstaatliches Recht der Wirtschaft.	Collin, Peter ▾	PI ▾					
12	proj-002	Nichtstaatliches Recht der Wirtschaft.	Wolf, Johanna ▾	researcher ▾					
13	proj-002	Nichtstaatliches Recht der Wirtschaft.	Wagner, Andreas ▾	researcher ▾					
14	proj-002	Nichtstaatliches Recht der Wirtschaft.	Solonets, Polina ▾	researcher ▾					
15	proj-002	Nichtstaatliches Recht der Wirtschaft.	Ebbertz, Matthias ▾	researcher ▾					
16	proj-002	Nichtstaatliches Recht der Wirtschaft.	Vesper, Tim-Niklas ▾	researcher ▾					
17	proj-002	Nichtstaatliches Recht der Wirtschaft.	Michel, Lisa ▾	student-assistant ▾					
18	proj-002	Nichtstaatliches Recht der Wirtschaft.	Gödde, Ben ▾	student-assistant ▾					
19	proj-002	Nichtstaatliches Recht der Wirtschaft.	Wolf, Paulina ▾	student-assistant ▾					

Pilot-Modell → unser Hybrid

Pilot-Experiment

- Separate Personen-Tabelle ✓
- ORCIDs und IDs ✓
- Beziehungen als Separator-Listen ✗
- 9 Personen in einer Zelle ✗
- Sheets kann nicht filtern ✗

Unser Hybrid-Modell

- Separate Personen im Long-Tab ✓
- ORCIDs in `authority` ✓
- Eine Zeile pro Beziehung ✓
- Filtern, sortieren, zählen ✓
- Sheets kann damit arbeiten ✓

Die guten Ideen übernehmen, die Umsetzung an Sheets anpassen.

Der core-Tab: das Wide-Format in Aktion

	A	B	C	D	E	
1	project_id ▾	title_de ▾	title_en ▾	title_es ▾	slug ▾	descripti
2	proj-001	Die Schule von Salamanca. Eine digitale Quellense	The School of Salamanca. A Digital Collection of S	La Escuela de Salamanca. Colección digital de fue	salamanca-school	Die digit
3	proj-002	Nichtstaatliches Recht der Wirtschaft. Die normat	Non-state law of the economy. The normative ord	Derecho económico no estatal. El orden normativ	metall-industry	In der Qu
4	proj-003	Historisches Wörterbuch des kanonischen Rechts	Historical dictionary of canon law in hispanic Ame	Diccionario Histórico de Derecho Canónico en His	canon-dictionary	Historisc
5	proj-004	Gonzalo Fernández de Oviedo	Gonzalo Fernández de Oviedo	Gonzalo Fernández de Oviedo	gonzalo-oviedo	Er war da
6	proj-005	Verhandelte Ordnung. Arbeitsbeziehungen in der V	Negotiated Order. Labour Relations in the Weimar	Orden negociado. Las relaciones laborales en la R	verhandelte-ordnung	Die Diss
7						
8						
9						
10						
11						
12						
13						
14						
15						
16						
17						
18						
19						
20						
21						
22						
23						
24						
25						

Wide vs. Long: zwei Formate

Wide (breit)

- Eine Zeile pro Projekt
- Viele Spalten nebeneinander
- Gut für **Singletons**: Felder mit genau einem Wert (Titel, Status, Jahr)
- Beispiel: unser `core` -Tab

Long (lang, tidy)

- Eine Zeile pro Beziehung/Fakt
- Wenige Spalten, viele Zeilen
- Gut für **Many-to-Many**: Felder mit vielen Werten (Personen, Keywords)
- Beispiel: unser `people` -Tab

Menschen lesen wide gut. Sheets-Funktionen (Filter, Sortieren, Gruppieren) funktionieren auf long gut. Unser Hybrid bedient beide.

Die Faustregel

Wenn ein Feld ...	Dann ...
genau einen Wert hat (Titel, Status, Startjahr)	Wide im <code>core</code> -Tab
1-N Werte, N klein und fix (max. 3 URLs)	Wide mit <code>url1</code> , <code>url2</code> , <code>url3</code>
unbegrenzt viele Werte (Personen, Keywords)	Long in einem eigenen Tab

Faustregel: *Kann ein Projekt mehrere davon haben? Dann long.*

Unser Hybrid-Modell

Tab	Format	Inhalt
core	wide	Ein Projekt pro Zeile, alle Singletons
people	long	Eine Person-Rolle pro Zeile
institutions	long	Eine Institution-Beziehung pro Zeile
subjects	long	Ein Thema pro Zeile
regions	long	Eine Region pro Zeile
keywords	long	Ein Keyword pro Zeile

Warum „Hybrid“? Weil wir nicht „entweder wide oder long“ wählen, sondern beides nebeneinander nutzen. Jedes Feld kommt in das Format, das zu seiner Kardinalität passt.

Gruppieren: Long lesen, als wäre es wide

	A	B	C	D	E	F	G	H	I	J
1	project_id	title_de	keyword							
	project_id: proj-001 Anzahl: 6 Ausgefüllt: 100%									
2	proj-001	Die Schule von Salamanca. Eine digitale Quellensa	kooperation							
3	proj-001	Die Schule von Salamanca. Eine digitale Quellensa	digitale-geisteswissenschaften							
4	proj-001	Die Schule von Salamanca. Eine digitale Quellensa	quellen							
5	proj-001	Die Schule von Salamanca. Eine digitale Quellensa	iberische-welten							
6	proj-001	Die Schule von Salamanca. Eine digitale Quellensa	normativität							
7	proj-001	Die Schule von Salamanca. Eine digitale Quellensa	religion							
	project_id: proj-002 Anzahl: 2 Ausgefüllt: 100%									
8	proj-002	Nichtstaatliches Recht der Wirtschaft. Die normati	digitale-geisteswissenschaften							
9	proj-002	Nichtstaatliches Recht der Wirtschaft. Die normati	sonderordnungen							
	project_id: proj-003 Anzahl: 3 Ausgefüllt: 100%									
10	proj-003	Historisches Wörterbuch des kanonischen Rechts	iberische-welten							
11	proj-003	Historisches Wörterbuch des kanonischen Rechts	normativität							
12	proj-003	Historisches Wörterbuch des kanonischen Rechts	religion							
	project_id: proj-004 Anzahl: 2 Ausgefüllt: 100%									
13	proj-004	Gonzalo Fernández de Oviedo	transmedia-historytelling							

Was wir aus dem Pilot-Experiment gelernt haben

1. **Relationales Denken ist wertvoll.** Eigene IDs, ORCIDs, Lookup-Tabellen. Das behalten wir.
2. **Separator-Listen passen nicht zu Sheets.** Die `creator`-Spalte mit 9 Pipe-getrennten Einträgen ist für Menschen und für Sheets unbrauchbar.
3. **Sheets ist ein Editor, nicht eine Datenbank.** Das Datenmodell muss für Menschen geschrieben sein. Die Mächtigkeit (Joins, Enrichment) entsteht zur Build-Zeit in Claude Code.

Modellierung ist ein Gespräch, kein einmaliger Wurf.

Zwei KIs im gleichen Sheet: eine Lehrgeschichte

In der Workshop-Vorbereitung haben zwei KIs parallel an diesem Sheet gearbeitet:

- **Claude** über die „Claude in Chrome“-Browser-Extension (steuert einen sichtbaren Chrome-Tab: navigiert, klickt, liest Zellen, macht Screenshots)
- **Gemini-in-Sheets** als Sidebar im Google-Workspace (nativ eingebaut)

Vier Iterationen: drei liefen gut, eine spektakulär schief. Die schief laufende lehrt am meisten.

Warum ich euch das zeige: Ihr werdet mit KIs arbeiten. Ihr müsst wissen, **wann ihr ihnen glauben könnt** und wann nicht.

Iteration 4: Was Gemini gemeldet hat

Auftrag: „Füge in den 5 Long-Tabs je eine neue Spalte `title_de` mit einer `XLOOKUP`-Formel ein.“

Geminis Self-Report:

„Ich habe die neue Spalte 'Title De' erfolgreich in die 5 gewünschten Tabs eingefügt. In den Blättern people, institutions, subjects, regions und keywords wurde jeweils eine neue Spalte B eingefügt. Die Formel `=XLOOKUP(A2, core!$A$2:$A$6, core!$B$2:$B$6, '')` wurde eingetragen. Die Validierungsregeln blieben erhalten.“

Klingt überzeugend. Klingt fertig. Klingt richtig.

Was im Sheet tatsächlich stand

Nur ein Tab (`people`) war angefasst, darin **acht** neue Spalten:

A	B	C	D	E	F	G	H
project_id	Title De	Column 10	Title De	Title De	Column 7	Column 6	Column 5

Alle 26 Datenzeilen in den neuen Spalten: `#ERROR!` . `role` war aus dem Viewport verschoben. **Reparatur:** `Ctrl+Z` × 20.

Geminis Bericht war eine **plausible Beschreibung des erwarteten Ergebnisses** – klassische LLM-Halluzination: detailliert, kohärent, falsch.

Drei Merksätze für den Umgang mit KI

Regel	Heißt konkret
Verifiziere immer.	Nie einem KI-Bericht glauben ohne das Artefakt selbst zu prüfen. Im Sheet: <code>Ctrl+End</code> (springt zur letzten benutzten Zelle – entlarvt heimliche Extra-Spalten/-Zeilen), Filter auf Verdachts-Werte. Im Code: <code>git diff</code> , Unit-Tests (kann Claude Code selbst schreiben).
Klein statt komplex.	Lieber 5 atomare Aufträge als einen großen. Jeder Schritt ist dann einzeln prüfbar. Komplexe Mehrschritt-Aufträge scheitern leiser.
Ein Bericht ist kein Beweis.	<i>„Ich habe X erledigt“</i> ist eine Behauptung. Der Beweis liegt im Artefakt, nicht im Report darüber.

Diese drei Regeln gelten für jede KI, nicht nur für Gemini. Auch für Claude Code.

Block 2: Datenmodell mit Claude Code

Wie wir in Block 2 arbeiten

Ab jetzt: live in Claude Code. Ihr promptet selbst.

Technisch läuft:

- Claude Code ($\geq 2.0.73$) auf euren Rechnern (aus WS2)
- Das Repo `legal-history-hub/` ist geklont
- Google Chrome ist offen, im Google Sheet eingeloggt
- Die Extension „**Claude in Chrome**“ ist im Chrome Web Store installiert
- Chrome-Integration aktiviert: im **Terminal** mit `/chrome`, in der **VS-Code-Extension** per `@browser` -Mention im Prompt-Feld

Claude Code steuert euren Chrome-Tab

Nativ eingebaute Integration (kein MCP): Sheet öffnen, Zellen lesen, klicken, Screenshots machen. Sichtbar in einem echten Browser-Fenster.

Wenn es hakt („Erweiterung nicht erkannt“):

- Terminal: `/chrome` → *Erweiterung erneut verbinden*
- VS Code: `@browser` erneut im Prompt senden
- Notfalls Claude Code und Chrome neu starten

Ziel: Ihr erlebt, dass Claude Code euer Datenmodell versteht, erklären kann, und Fehler findet, die Sheets allein nicht sieht.

Übung 1: Das Modell erklären lassen

Promptet Claude Code nacheinander:

1. *„Öffne unser Google Sheet im Chrome und erkläre das Hybrid-Modell in eigenen Worten.“*
2. *„Was ist der Unterschied zwischen dem core-Tab (wide) und dem people-Tab (long)? Nimm proj-001 als Beispiel.“*
3. *„Warum steht in den Long-Tabs das title_de in jeder Zeile, obwohl es im core schon vorkommt?“*

Ziel: Claude Code **versteht und erklärt** das Modell, es führt nicht nur Code aus.

Übung 2: Ein neues Feld einbauen

Schlagt ein Feld vor (z.B. Kontaktperson, DOI, Projektleitung).

Dann fragt Claude Code: „*Ich möchte [dieses Feld] pro Projekt speichern. Wohin soll das?*“

Die Fragen, die Claude Code stellen sollte:

Frage	Antwort → Konsequenz
Wie viele Kontaktpersonen pro Projekt?	Genau eine → wide in <code>core</code>
Können es auch mehrere sein?	Ja → long in <code>people</code> mit Rolle <code>contact</code>
Braucht die E-Mail eigenen Platz?	→ Neue Spalte in <code>core</code> , oder in <code>authority</code>

Lernziel: Die Faustregel aus Block 1 an einem echten Fall anwenden.

Übung 3: Fehler finden lassen

Im [Übungs-Sheet](#) sind **absichtliche Fehler** versteckt.

Promptet Claude Code:

- *„Prüfe die Tabs gegen sich selbst. Finde Inkonsistenzen.“*
- *„Zeig mir jede Zeile im people-Tab, deren role nicht in vocabulary[person_roles] vorkommt.“*
- *„Gibt es project_ids in den Long-Tabs, die im core nicht existieren?“*
- *„Gibt es Keywords, die in keywords stehen, aber in authority fehlen?“*

Die vier versteckten Fehler

#	Fehler	Typ
1	<code>role = "mitarbeiter"</code> statt <code>researcher</code>	Unbekannter Enum-Wert
2	<code>project_id = "proj-099"</code> in <code>people</code>	Dangling Foreign Key
3	Keyword <code>natural-law</code> nicht in authority	Fehlende Anreicherung
4	<code>proj-006</code> ohne <code>title_de</code>	Leeres Pflichtfeld

Claude Code findet sie, erklärt sie. Ihr behebt sie im Sheet, Claude prüft erneut.

Fehler finden, beheben, erneut prüfen. Dieser Zyklus ist der Kern des Workflows.

Block 2: Was mitnehmen?

- Claude Code kann euer Datenmodell **lesen und erklären**, nicht nur Code ausführen.
- Die **wide-vs-long-Entscheidung** ist ein wiederholbarer Prozess: „Wie viele Werte? Einer oder viele?“
- **Referentielle Integrität** (passen die IDs zusammen?) ist etwas, das Claude Code prüfen kann, Sheets allein aber nicht.
- **Die drei Merksätze aus Block 1 gelten weiter:** verifizieren, klein arbeiten, Bericht ist kein Beweis.

Pause (10-15 min)

Danach: Sheet als CMS aufbauen.

Block 3: Google Sheets als CMS

Jetzt richten wir unser Sheet so ein, dass es als **Editor** gut funktioniert: Dropdowns, Validierung, Ansichten.

Ziel: Das Sheet kennt die Regeln des Datenmodells und hilft beim Einhalten.

Schritt 1: Tabs zu Tables konvertieren

Format → In Tabelle konvertieren

Was Tables bringen:

- **Spalten-Typen:** Number akzeptiert keinen Text, Date kein Freitext
- **Auto-Expansion:** Neue Zeilen erben Formatierung, Validierung, Formeln
- **Benannte Referenzen:** Die Tabelle heißt `core`, `people`, etc.
- **Filter im Header:** Eingebaut, ohne extra Filter-Ansicht

Tables sind kein Styling. Sie sind ein **Datenobjekt** mit Regeln.

<https://support.google.com/docs/answer/14239833>

Structured References: Formeln, die man lesen kann

Klassisch (fragil):

```
=VLOOKUP(A2; core!A:B; 2;  
        FALSE)
```

Die `2` bedeutet „zweite Spalte“. Bricht bei jeder Spalten-Einfügung.

Die Syntax `core[project_id]` heißt wörtlich: „*die Spalte `project_id` der Tabelle `core`*“.

Mit Tables (stabil):

```
=XLOOKUP(A2;  
        core[project_id];  
        core[title_de]; "")
```

Lesbar wie ein Satz. Überlebt Spalten-Einfügungen.

Wie tippt man `core[project_id]`?

Nicht auswendig. Sheets hilft.

1. Formel anfangen: `=XLOOKUP(A2;`
2. Tabellennamen tippen: `co...` → Sheets schlägt `core` vor
3. Eckige Klammer `[` → Sheets zeigt alle Spalten der Tabelle als Liste
4. Spalte auswählen → Sheets schließt die Klammer `]` automatisch

Voraussetzung: Der Tab muss vorher mit `Format → In Tabelle konvertieren` zu einer Table gemacht worden sein. Sonst kennt Sheets den Namen nicht.

Das Semikolon-Problem

Achtung: Deutsches Sheets verwendet **Semikolon** als Argument-Trenner (weil Komma das Dezimaltrennzeichen ist, z.B. `3,14`).

Online-Beispiele und KI-Output nutzen fast immer Kommas. Kopieren → `#ERROR!` .

Falsch (englisch)	Richtig (deutsch)
<code>=XLOOKUP(A2, core[project_id], core[title_de], "")</code>	<code>=XLOOKUP(A2; core[project_id]; core[title_de]; "")</code>

Bei `#ERROR!` zuerst Kommas durch Semikolons ersetzen.

Schritt 2: title_de als Lesehilfe

In jedem Long-Tab eine Formel-Spalte `title_de` nach `project_id`:

```
=XLOOKUP(A2; core[project_id]; core[title_de]; "")
```

- Neue Zeile einfügen → `project_id` eingeben → Titel erscheint sofort
- Titel im `core` ändern → aktualisiert sich überall

`core.title_de` bleibt **Source of Truth**, alle anderen Vorkommen sind Abbildungen davon.

Schritt 2b: Wenn Auto-Expansion nicht greift

Google Sheets Tables kopieren Formeln normalerweise automatisch in neue Zeilen. Falls nicht:

Ctrl+D (Fill Down) ist der universelle Fallback.

1. Zelle mit der funktionierenden Formel markieren
2. Bereich nach unten bis zur letzten Zeile erweitern (**B2:Bn**)
3. **Ctrl+D** drücken

Die Formel verteilt sich auf alle markierten Zellen, mit an die Zeile angepassten Referenzen. Funktioniert auch außerhalb von Tables.

Schritt 3: Drei Stufen der Validierung

Hinter jedem Dropdown steht eine **Quelle**. Unser Sheet hat drei:

Stufe	Tab	Rolle
vocabulary	vocabulary	Geschlossene Enum-Listen (Rollen, Status)
authority	authority	Normdatei mit PIDs (ORCID, GND, ROR)
_helpers	_helpers	Gefilterte Sichten pro Typ, an die Dropdowns gebunden sind

vocabulary ist Vorschrift, authority ist Nachschlagewerk, _helpers ist die Brille.

Authority wird **nie direkt** an ein Dropdown gebunden, sondern immer über `_helpers`.
Warum: nächste Folie.

Eingabe ablehnen vs. Warnung anzeigen

Im Datenvalidierungs-Panel gibt es zwei Modi:

Eingabe ablehnen

Rotes X, Wert wird **nicht** gespeichert.

Für echte Enums, wo falsche Werte unmöglich sein sollen: `status` , `role` .

Faustregel: *Ablehnen* nur bei echten Enums aus `vocabulary` . Sonst immer *Warnung*, damit das Sheet wachsen darf, ohne zu blockieren.

Warnung anzeigen

Rote Ecke, Wert **wird** gespeichert.

Für Fremd-IDs und Authority-Bindungen, wo neue Werte später nachgetragen werden: `person` , `keyword` , `project_id` .

Warum drei Stufen statt einer?

Problem: `authority` enthält alles durcheinander – Personen, Institutionen, Keywords, Regionen. Ein Dropdown direkt darauf würde alle Typen vermischen.

Lösung: `_helpers` filtert:

```
=FILTER(authority[label]; authority[type]="person")
```

`person_labels` zeigt nur Personen, `keyword_labels` nur Keywords. Dank **Spill Range** wächst die Liste mit der Quelle – ohne Nachziehen.

Dropdown-Architektur im Überblick

Zielzelle	Bindung	Stufe	Modus
<code>core.status</code>	<code>vocabulary[status_values]</code>	vocabulary	ablehnen
<code>people.role</code>	<code>vocabulary[person_roles]</code>	vocabulary	ablehnen
<code>people.person</code>	<code>_helpers[person_labels]</code>	_helpers	warnen
<code>institutions.institution</code>	<code>_helpers[institution_labels]</code>	_helpers	warnen
<code>institutions.relation</code>	<code>vocabulary[institution_relations]</code>	vocabulary	ablehnen
<code>subjects.subject</code>	<code>_helpers[subject_labels]</code>	_helpers	warnen
<code>keywords.keyword</code>	<code>_helpers[keyword_labels]</code>	_helpers	warnen
alle <code>project_id</code>	<code>core[project_id]</code>	Foreign Key	warnen

Warnung für alles, was wachsen darf. **Ablehnen** nur für echte Enums.

Dropdowns in Aktion

	A	B	C	D	E	F	G	H	I
1	project_id ▾	title_de ▾	person ▾	role ▾					
2	proj-001	Die Schule von Salamanca. Eine digita	Duve, Thomas ▾	PI ▾					
3	proj-001	Die Schule von Salamanca. Eine digita	Lutz-Bachmann, Matthias ▾	PI ▾					
4	proj-001	Die Schule von Salamanca. Eine digita	König, Florian ▾	researcher ▾					
5	proj-001	Die Schule von Salamanca. Eine digita	Birr, Christiane ▾	researcher ▾					
6	proj-001	Die Schule von Salamanca. Eine digita	Rico Carmona, Cindy ▾	researcher ▾					
7	proj-001	Die Schule von Salamanca. Eine digita	Solonets, Polina ▾	researcher ▾					
8	proj-001	Die Schule von Salamanca. Eine digita	Wagner, Andreas ▾	researcher ▾					
9	proj-001	Die Schule von Salamanca. Eine digita	Hugel, Marie-Astrid ▾	researcher ▾					
10	proj-001	Die Schule von Salamanca. Eine digita	Schweighöfer, Stefan ▾	project-coordinator ▾					
11	proj-002	Nichtstaatliches Recht der Wirtschaft.	Collin, Peter ▾	PI ▾					
12	proj-002	Nichtstaatliches Recht der Wirtschaft.	Wolf, Johanna ▾	researcher ▾					
13	proj-002	Nichtstaatliches Recht der Wirtschaft.	Wagner, Andreas ▾	researcher ▾					
14	proj-002	Nichtstaatliches Recht der Wirtschaft.	Solonets, Polina ▾	researcher ▾					
15	proj-002	Nichtstaatliches Recht der Wirtschaft.	Ebbertz, Matthias ▾	researcher ▾					
16	proj-002	Nichtstaatliches Recht der Wirtschaft.	Vesper, Tim-Niklas ▾	researcher ▾					
17	proj-002	Nichtstaatliches Recht der Wirtschaft.	Michel, Lisa ▾	student-assistant ▾					
18	proj-002	Nichtstaatliches Recht der Wirtschaft.	Gödde, Ben ▾	student-assistant ▾					
19	proj-002	Nichtstaatliches Recht der Wirtschaft.	Wolf, Paulina ▾	student-assistant ▾					

Warum waren die Dropdowns hartkodiert?

Gemini-in-Sheets hat die Dropdowns in der Vorbereitung gebaut. Ihr erinnert euch an die Halluzinations-Story aus Block 1 (Gemini meldet Erfolg, wo keiner war). Hier kommt eine **andere Gemini-Schwäche** dazu: Dropdowns *aus einem Bereich* kann es strukturell nicht. Nur **Einzelwerte-Listen**.

Google dokumentiert das selbst:

„Selecting from a range is not supported in Gemini's dropdown creation feature.“

Das ist kein Bug, sondern ein **strukturelles Limit**. Dynamisches Problem (Halluzination) vs. statisches Problem (Feature-Grenze). Beide muss man kennen. Deshalb bauen wir die Dropdowns jetzt auf dynamische Bereichs-Bindung um, manuell.

Der 10-Sekunden-Check

Wie erkennst du, ob ein Dropdown richtig gebunden ist?

In einer Table (seit Tables-Konvertierung): am Spaltenkopf auf das Typ-Symbol klicken → *Spaltentyp bearbeiten*. Dort siehst du, ob der Spaltentyp *Drop-down* oder *Drop-down (aus Bereich)* ist.

Du siehst ...	Bedeutung
„Drop-down“ (Einzelwerte-Liste)	Hartkodiert. Umbauen!
„Drop-down (aus Bereich)“ mit Referenz wie <code>vocabulary[person_roles]</code>	Dynamisch gebunden. Richtig.

Alternativ außerhalb von Tables: *Daten* → *Datenvalidierung* zeigt dasselbe. Diesen Check könnt ihr auf jedes Sheet anwenden.

Datenansichten: Long-Tabs im Fokus

Daten → Datenansichten → Neue Datenansicht erstellen

Filter auf `project_id = "proj-001"` → speichern als „*proj-001 people*“

Was das bringt:

- Du siehst **nur die Zeilen** für ein Projekt
- Du editierst **in Fokus**, ohne andere Zeilen zu stören
- Jeder Nutzer kann **eigene Ansichten** haben, gleichzeitig

Filter View oder Gruppieren?

Ziel	Bessere Wahl
Ein Projekt bearbeiten (nur dessen Zeilen)	Filter View
Alle Projekte im Überblick mit Anzahl	Gruppieren-Ansicht
Projekt-übergreifend auswerten (alle PIs)	Gruppieren nach <code>role</code>

Beide sind **nutzer-lokal** und stören sich nicht gegenseitig.

Bedingte Formatierung

	A	B	K	L	M	N	O	P	Q	R	S	T
1	project_id ▾	title_de ▾	🔍 status ▾	url1 ▾	url2 ▾	url3 ▾	date_added ▾	🔍 verified ▾				
2	proj-001	Die Schule von Salamanca. Eine digitale Quellens	active ▾	https://www.lhlt.mpg.de/gemeinschaftsprojekt/di	https://www.salamanca.school/		27.02.2026	v ▾				
3	proj-002	Nichtstaatliches Recht der Wirtschaft. Die normat	completed ▾	https://www.lhlt.mpg.de/3455831/jp-collin-non-st	https://metallindustrie.lhlt.mpg.de/		19.03.2026	v ▾				
4	proj-003	Historisches Wörterbuch des kanonischen Rechts	active ▾	https://www.lhlt.mpg.de/research-project/historic	https://dch.hypotheses.org/		19.03.2026	v ▾				
5	proj-004	Gonzalo Fernández de Oviedo	active ▾	https://www.lhlt.mpg.de/3285537/02-damler-gon			19.03.2026	v ▾				
6	proj-005	Verhandelte Ordnung. Arbeitsbeziehungen in der V	completed ▾	https://www.lhlt.mpg.de/2524472/jp-ebbertz-worl			19.03.2026	v ▾				
7												
8												
9												
10												
11												
12												
13												
14												
15												
16												
17												
18												
19												
20												
21												
22												
23												
24												
25												
26												
27												
28												
29												
30												
31												
32												
33												
34												

Bedingte Formatierung einrichten

Beispiel: abgeschlossene Projekte sollen die ganze Zeile grau zeigen.

1. Datenbereich markieren: in die Tabelle klicken, `Ctrl+A` erfasst alles (mit Tables). Sonst manuell, z.B. `A2:N50`.
2. `Format → Bedingte Formatierung`
3. Rechts im Panel: „**Benutzerdefinierte Formel ist**“
4. Formel eingeben: `= $F2="completed"` (Spalte F = `status`)
5. Hintergrundfarbe wählen (grau), speichern

Warum das \$ in =\$F2="completed" ?

- \$F heißt: *immer* Spalte F prüfen, egal in welcher Spalte die Zelle liegt
- 2 ohne \$ heißt: pass die Zeile an, wenn die Regel auf andere Zeilen zieht

Ergebnis: Zeile 2 wird grau, wenn `F2 = "completed"`. Ganze Zeile, nicht nur die Status-Zelle.

Ohne \$F würde die Regel in Spalte B auf Spalte B schauen – dort steht kein `status`, nichts würde grau.

Vom Editor zum Hub: wie kommen die Daten nach draußen?

Unser Sheet ist jetzt ein ordentliches CMS. Aber der Hub liest kein Google Sheet direkt – er braucht eine statische JSON-Datei.

Die Brücke: ein Python-Script, das via Google Sheets API liest, joined, validiert und `projects.json` schreibt. Claude Code hat es gebaut, Claude Code ruft es auf.

Nächste Folien: was die Sheets API ist, und wie die Pipeline konkret aussieht.

Was ist die Sheets API?

Eine **API** (Application Programming Interface) ist eine Schnittstelle, über die Programme miteinander reden. Die **Google Sheets API** lässt Claude Code direkt aufs Sheet zugreifen, ohne Browser-Klicks.

Chrome-Integration (Block 2)	Sheets API (Pipeline)
Claude steuert den Browser	Claude fragt den Sheets-Server direkt
Gut für: lesen, Screenshots, einzelne Edits	Gut für: Massen-Lesen, Build-Pipeline
Langsam, visuell	Schnell, strukturiert
„Claude in Chrome“ + <code>/chrome</code> bzw. <code>@browser</code>	Credentials vorab vorbereitet

Beide haben ihren Platz. Die API ist der Profi-Weg für den Build.

Hands-on: Die Pipeline einmal durchspielen

```
Google Sheets (9 Tabs)
  ↓ Claude Code führt scripts/build-hub-data.py aus
  ↓ Script liest per Sheets API, joined, validiert
  data/projects.json (nested, enriched)
  ↓ git add, commit, push
GitHub Pages (Hub live)
```

1. Neue Person in Sheets eintragen
2. Claude Code: „Baue `projects.json` neu“ → führt das Script aus
3. Ergebnis prüfen, committen, pushen
4. Hub zeigt die neue Person

Script vorab fertig. In Block 4 schauen wir rein.

Block 3: Was mitnehmen?

- **Tables** sind Datenobjekte, nicht Styling. Structured References machen Formeln lesbar.
- **Drei Stufen:** vocabulary (Vorschrift), authority (Nachschlagewerk), _helpers (Brille).
- **Ablehnen vs. Warnung:** Ablehnen nur bei Enums, Warnung überall sonst.
- **Datenansichten** machen Long-Tabs alltagstauglich.
- **Die Pipeline:** Sheets → Claude Code (API) → `projects.json` → Hub.

Block 4: Skills und Plugins

Claude Code kann mehr als einzelne Prompts. **Skills** und **Plugins** machen wiederkehrende Aufgaben wiederholbar.

Was sind Skills?

- Wiederverwendbare Befehle, die Claude Code mit `/` aufruft
- **Prompt-Wrapper**: ein Wort aktiviert eine strukturierte Anleitung, die Claude mit **Urteil** ausführt – nicht als Makro, sondern situativ
- Ein Skill ist ein **Ordner** mit einer `SKILL.md`-Datei darin (kann weitere Hilfsdateien enthalten)

Referenz: <https://code.claude.com/docs/en/skills.md>

Skills: zwei Orte

legal-history-hub/ .claude/skills/ validate-data/ SKILL.md enrich-authority/ SKILL.md	← projekt-lokal, im Repo versioniert
~/.claude/skills/ promptotyping/ SKILL.md	← global, über alle Projekte

Lokal für Hub-spezifisches, global für alles, was ihr in mehreren Projekten braucht.

Skills installieren: `npx skills`

Skills liegen als Ordner in GitHub-Repos. Die Vercel-Labs-CLI zieht sie auf euren Rechner:

```
npx skills add anthropics/skills --skill skill-creator -g -a claude-code  
npx skills add DigitalHumanitiesCraft/promptotyping-skill -g -a claude-code
```

- `-g` = global (`~/.claude/skills/`), ohne `-g` projekt-lokal
- `-a claude-code` = Ziel-Agent (auch `cursor` , `codex` , ...)
- `npx skills list` / `update` für Übersicht und Aktualisierung

Script oder Skill?

Nicht jedes Problem ist ein Skill-Problem.

	Python-Script	Skill (<code>SKILL.md</code>)
Input → Output	deterministisch, gleich bei gleichem Input	variabel, kontextabhängig
Fehlermodus	Exception, Stacktrace	Halluzination möglich
Automatisch testbar (CI*)	ja	nein
Anpassung	Code editieren, testen	Prompt editieren
Hub-Beispiel	<code>build-hub-data.py</code> (Sheet → JSON)	<code>/explain-model</code> , <code>/enrich-authority</code>

Regel: *Gleicher Input → gleiches Ergebnis = Script. Claudes Urteil ist Teil des Werts = Skill.*

*CI = Continuous Integration. Automatische Test-Läufe bei `git push` . Scripts laufen dort, Skills nicht.

Live-Demo: /promptotyping

Wir rufen `/promptotyping` auf und beobachten.

Der Skill liest die Promptotyping-Dokumente des Projekts und schlägt vor, welcher Dokumenttyp als Nächstes sinnvoll wäre (RESEARCH, REQUIREMENTS, DESIGN, JOURNAL).

Dann öffnen wir die Skill-Datei im Editor:

„Das ist ein Markdown-Dokument mit Anweisungen. Claude Code liest es und führt die Schritte aus.“

Ihr könnt also eigene Skills schreiben. Alles, was ihr in natürlicher Sprache formulieren könnt, kann ein Skill werden.

Voraussetzung: `/skill-creator`

Skills schreibt man nicht einfach in eine Markdown-Datei. Anthropic liefert dafür einen **Meta-Skill**: `/skill-creator`. Er weiß, wie ein guter Skill aussieht.

Er kümmert sich um:

- saubere Frontmatter (`name` , `description`)
- eine präzise `description` – daran erkennt Claude, wann der Skill feuern soll
- progressive disclosure (Details in Unterdateien, nicht alles in `SKILL.md`)

Ohne `/skill-creator` schreibt Claude ein Markdown, das *aussieht* wie ein Skill. Mit `/skill-creator` wird es einer.

Gemeinsam: /validate-data live bauen

Wir bauen einen **Skill**, kein Script. Der Unterschied:

- Ein Script listet Fehler roh in `stdout`
- Ein Skill **erklärt** Findings auf Deutsch, **priorisiert**, **schlägt Fixes vor**

Der Skill-Vorteil: natürlichsprachige Erklärung für die Editor-Arbeit, nicht nur Fehler-Codes.
Ein reiner Checker wäre besser ein Python-Script.

/validate-data: der Ablauf

1. `/skill-creator` aufrufen. Auf die Rückfrage beschreiben: „Ein Skill `/validate-data`, der das Sheet über die Chrome-Integration liest, alle `project_id` in den Long-Tabs gegen `core` prüft, `role` und `relation` gegen `vocabulary`, und in klaren deutschen Sätzen erklärt, was nicht stimmt und wie ich es fixe.“
2. Claude Code schreibt `.claude/skills/validate-data/SKILL.md`
3. Wir lesen die Datei gemeinsam
4. Wir rufen `/validate-data` auf – Claude Code erkennt neue Skills live, ohne Neustart
5. Verfeinern: „Zeig bei jedem Fund auch die konkrete Zeile, das Projekt und einen Fix-Vorschlag.“

Skills und Scripts für den Hub

Typ	Name	Was es tut	Status
Script	<code>build-hub-data.py</code>	Sheets → <code>projects.json</code> (Join + Enrichment)	vorhanden
Skill	<code>/explain-model</code>	Erklärt Modellstand auf Deutsch	Idee
Skill	<code>/add-project</code>	Geführter Dialog für neue Projekte	Idee
Skill	<code>/validate-data</code>	Findings + Fix-Vorschläge	gleich
Skill	<code>/enrich-authority</code>	ORCID/GND-Lücken mit Approval-Loop	Idee

Brainstorming: Welchen Skill würdet ihr als Nächstes brauchen?

MCP-Plugins: neue Fähigkeiten

Skills sagen Claude Code was es tun soll. **MCP-Plugins** geben Claude Code neue **Werkzeuge**.

MCP = **Model Context Protocol**, ein Standard für den Draht zwischen KI und externen Diensten.

Beispiele: **Google-Sheets-MCP** (API-Zugriff aufs Sheet), **ORCID-API-Plugin** (Forscher-Kennungen), **GitHub-MCP** (Issues/PRs).

Die Chrome-Integration aus Block 2 ist **kein** MCP-Plugin, sondern nativ eingebaut. Ihr müsst Plugins nicht selbst bauen – wichtig ist nur zu wissen, dass es sie gibt.

CLAUDE.md: das Projektgedächtnis

Claude Code liest bei jedem Start `CLAUDE.md` im Projekt-Root – die Bedienungsanleitung für dieses Projekt.

Darin steht:

- Welche Tabs das Sheet hat
- Wie das Hybrid-Modell aufgebaut ist
- Welche Konventionen gelten
- Welche Skills verfügbar sind

Wenn Claude Code euer Modell „kennt“, liegt das an dieser Datei. Wenn etwas fehlt oder veraltet ist: `CLAUDE.md` ergänzen.

Block 4: Was mitnehmen?

- **Skills** = Prompt-Wrapper für Workflows mit Claudes Urteil. `.md` -Dateien in `.claude/skills/`.
- **Scripts** = deterministische Automatisierung (Python, Shell). Claude Code schreibt und pflegt sie, aber sie sind **keine Skills**.
- **Faustregel**: gleicher Input, gleicher Output → Script. Urteil nötig → Skill.
- **MCP-Plugins** = neue Werkzeuge (APIs, Sheets-Zugriff). Existieren, ihr müsst sie nicht bauen.
- **CLAUDE.md** = Projektgedächtnis. Claude Code „weiß“ nur, was dort steht.

Block 5: Zusammenfassung

Learnings

- Ein Datenmodell als **Hybrid aus wide + long** beschreiben und begründen
- Die Faustregel anwenden: Singleton → wide, Many-to-Many → long
- Dropdowns einrichten, die auf `vocabulary` oder `_helpers` verweisen
- Filter Views und Gruppieren-Ansichten pro Projekt anlegen
- Mit Claude Code ein Modell **lesen, erklären lassen, erweitern, validieren**
- Den Flow **Sheets** → **Claude Code (API)** → `projects.json` → **Hub** selbst durchführen
- Skills als wiederverwendbare Arbeitsanweisungen erkennen und **eigene Ideen formulieren**
- KI-Ergebnisse **verifizieren**, statt Self-Reports zu glauben

Das größere Bild

Sheets ist der Editor, nicht das Modell.

Die Mächtigkeit (Joins, Enrichment, PIDs) entsteht zur Build-Zeit in Claude Code.

Das Datenmodell ist kein einmaliger Wurf.

Pilot-Experiment → Hybrid-Modell: genau so ein Gespräch. Zwischenschritte sind keine Fehler.

Kontrolliertes Vokabular + Authority-Lookup ist die eine Stelle, an der Normalisierung Sinn macht. Alles andere darf flach und lesbar sein.

An Validierung denken! Jedes KI-Ergebnis gegen das Artefakt prüfen – Sheet-Inhalt, Git-Diff, Dateistand. Self-Reports sind Behauptungen, nicht Beweise.

Ausblick: WS4 – WS6

Workshop	Thema
WS4	Claude-Infrastruktur im Arbeitsalltag. Context Engineering, Claude Desktop, Projects, Artifacts, Memory, die Produktlandschaft (Excel, Word, PowerPoint, Chrome, Code). Welches Werkzeug wann.
WS5	Offen – Themen ergeben sich aus WS4 und eurem Bedarf.
WS6	Offen – Feinschliff, Anbindung an Kollaborationsprozesse oder ein eigener Use Case.

In jedem Workshop: immer mit Claude Code, immer iterativ.

Danke!

Ressourcen:

- Tutorial-Lektion 3: *Das Datenmodell verstehen*
- Cheat Sheet: Faustregeln, Drei-Stufen-Dropdowns, Top-10-Prompts
- `CLAUDE.md` und `README.md` im Repo

Fragen? Ideen? Feedback?